

Multi-tenant SaaS Platform User Training Manual

Table of Contents

1. Introduction
2. System Overview
3. System Requirements
4. Installation Guide
5. Configuration
6. Running the Application
7. User Interface Guide
8. API Usage
9. Multi-tenancy Features
10. Authentication and Security
11. Troubleshooting
12. Frequently Asked Questions

Introduction

Welcome to the Multi-tenant SaaS Platform User Training Manual. This comprehensive guide will walk you through all aspects of installing, configuring, and using the SaaS Platform. The platform has been designed with a modular architecture, clear separation of concerns, and robust multi-tenancy features to provide a secure and scalable solution for your organization.

This manual is intended for system administrators, developers, and end-users who will be working with the SaaS Platform. It provides step-by-step instructions for setting up the environment, running the application, and utilizing all available features.

System Overview

The Multi-tenant SaaS Platform is built with a modern technology stack and follows best practices for software architecture. The system consists of two main components:

1. **Backend API:** A FastAPI-based RESTful API that handles data processing, authentication, and business logic.
2. **Frontend GUI:** A Streamlit-based user interface that provides an intuitive way to interact with the system.

Key Features

- **Multi-tenancy:** Support for multiple tenant organizations with data isolation

- **User Authentication:** Secure login, registration, and password management
- **Role-Based Access Control:** Different permission levels based on user roles
- **Activity Tracking:** Comprehensive logging of user activities
- **Usage Statistics:** Tracking of monthly active users and other usage metrics
- **Tenant Configuration:** Customizable settings for each tenant
- **API Documentation:** Interactive API documentation with Swagger UI

Architecture

The system follows a modular architecture with clear separation of concerns:

- **Models:** Database schema definitions using SQLAlchemy
- **Schemas:** Data validation using Pydantic
- **Routes:** API endpoints using FastAPI
- **Services:** Business logic layer
- **Dependencies:** Reusable dependency injection components
- **Middleware:** Request processing components
- **Utilities:** Helper functions and tools
- **Configuration:** Centralized settings management

System Requirements

Before installing the SaaS Platform, ensure your system meets the following requirements:

Hardware Requirements

- **CPU:** 2+ cores recommended
- **RAM:** 4GB minimum, 8GB recommended
- **Disk Space:** 1GB for application, additional space for database

Software Requirements

- **Operating System:** Linux, macOS, or Windows
- **Python:** Version 3.8 or higher
- **PostgreSQL:** Version 12 or higher
- **Redis:** Version 6 or higher (optional but recommended)

Network Requirements

- **Ports:** 8000 (API), 8501 (Streamlit UI), 5432 (PostgreSQL), 6379 (Redis)
- **Internet Access:** Required for package installation

Installation Guide

This section provides detailed instructions for installing the SaaS Platform from scratch.

Step 1: Clone the Repository

First, clone the repository to your local machine:

```
git clone https://github.com/sadeqobaid/saas_refactor.git
cd saas_refactor
```

Step 2: Set Up Python Environment

Create and activate a virtual environment:

```
# Create virtual environment
python -m venv venv

# Activate on Windows
venv\Scripts\activate

# Activate on macOS/Linux
source venv/bin/activate
```

Step 3: Install Dependencies

Install all required packages:

```
pip install fastapi uvicorn sqlalchemy pydantic python-jose[cryptography] passlib python-mul
```

Step 4: Set Up PostgreSQL

1. Install PostgreSQL if not already installed:

```
# Ubuntu/Debian
sudo apt update
sudo apt install postgresql postgresql-contrib

# macOS (using Homebrew)
brew install postgresql

# Windows
# Download and install from https://www.postgresql.org/download/windows/
```

2. Start PostgreSQL service:

```
# Ubuntu/Debian
sudo service postgresql start
```

```

# macOS
brew services start postgresql

# Windows
# PostgreSQL service should start automatically

```

3. Create a database for the application:

```

# Log in to PostgreSQL
sudo -u postgres psql

# Create database
CREATE DATABASE saas_platform;

# Create user (replace 'password' with a secure password)
CREATE USER saas_user WITH ENCRYPTED PASSWORD 'password';

# Grant privileges
GRANT ALL PRIVILEGES ON DATABASE saas_platform TO saas_user;

# Exit PostgreSQL
\q

```

Step 5: Set Up Redis (Optional but Recommended)

1. Install Redis:

```

# Ubuntu/Debian
sudo apt update
sudo apt install redis-server

# macOS
brew install redis

# Windows
# Download and install from https://redis.io/download

```

2. Start Redis service:

```

# Ubuntu/Debian
sudo service redis-server start

# macOS
brew services start redis

# Windows
# Start Redis server using the installed executable

```

Configuration

The SaaS Platform uses environment variables for configuration. These can be set in a `.env` file in the root directory.

Step 1: Create Environment File

Create a `.env` file in the root directory of the project:

```
touch .env
```

Step 2: Configure Environment Variables

Edit the `.env` file with your preferred text editor and add the following configuration:

```
# Database Configuration
DATABASE_URL=postgresql://saas_user:password@localhost:5432/saas_platform
DB_POOL_SIZE=5
DB_MAX_OVERFLOW=10

# Redis Configuration
REDIS_URL=redis://localhost:6379/0

# Security Configuration
SECRET_KEY=your-secret-key-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7
PASSWORD_RESET_TOKEN_EXPIRE_MINUTES=10

# Email Configuration
SMTP_SERVER=smtp.example.com
SMTP_PORT=587
SMTP_USERNAME=user@example.com
SMTP_PASSWORD=password
EMAIL_FROM=noreply@example.com
```

Replace the placeholder values with your actual configuration: - Replace `saas_user:password` with your PostgreSQL username and password - Generate a secure random string for `SECRET_KEY` - Update SMTP settings with your email server details

Step 3: Verify Configuration

To verify that your configuration is correct, you can run the validation script:

```
python validate.py
```

This script will check that your environment is properly configured and that the application can connect to the database and other services.

Running the Application

The SaaS Platform consists of two components that need to be run separately: the backend API and the frontend Streamlit application.

Running the Backend API

1. Ensure you're in the project root directory with the virtual environment activated.

2. Run the backend API using the provided run script:

```
python run.py
```

This will start the FastAPI application on <http://127.0.0.1:8000>.

3. You can access the API documentation at <http://127.0.0.1:8000/docs>, which provides an interactive interface to explore and test the API endpoints.

Running the Frontend GUI

1. Open a new terminal window.
2. Navigate to the project directory and activate the virtual environment:

```
cd path/to/saas_refactor
```

```
# Activate on Windows
```

```
venv\Scripts\activate
```

```
# Activate on macOS/Linux
```

```
source venv/bin/activate
```

3. Run the Streamlit application:

```
streamlit run streamlit_app.py
```

This will start the Streamlit application and automatically open it in your default web browser at <http://localhost:8501>.

Verifying the Installation

To verify that both components are running correctly:

1. Check that the API health endpoint returns a success response:

```
curl http://127.0.0.1:8000/health
```

You should see a JSON response with status “healthy” and component statuses.

2. Open the Streamlit UI in your browser at <http://localhost:8501> and verify that the login page loads correctly.

User Interface Guide

The Streamlit frontend provides an intuitive user interface for interacting with the SaaS Platform. This section guides you through the various screens and features available.

Navigation

The application has a sidebar navigation menu that changes based on your login status:

- **When not logged in:** Register, Login, Reset Password
- **When logged in:** Dashboard, Change Password, Logout

User Registration

To register a new user:

1. Navigate to the “Register” page from the sidebar.
2. Enter your email address.
3. Create a password that meets the security requirements:
 - Minimum 8 characters
 - At least 1 uppercase letter
 - At least 1 number
 - At least 1 special character
4. Click the “Register” button.
5. If successful, you’ll see a confirmation message and can proceed to login.

User Login

To log in to the application:

1. Navigate to the “Login” page from the sidebar.
2. Enter your email address.
3. Enter your password.
4. Click the “Login” button.
5. If successful, you’ll be redirected to the Dashboard.

Password Reset

If you forget your password:

1. Navigate to the “Reset Password” page from the sidebar.

2. In the “Request Reset” tab, enter your email address and click “Send Reset Link”.
3. Check your email for a reset token (in a production environment, this would be a link).
4. In the “Verify Token” tab, enter the token from your email and click “Verify Token”.
5. In the “Set New Password” tab, enter and confirm your new password, then click “Reset Password”.
6. If successful, you’ll see a confirmation message and can proceed to login with your new password.

Dashboard

The Dashboard is the main interface for logged-in users. It displays:

- User information
- Tenant information
- Recent activity
- Usage statistics

The content of the Dashboard varies based on the user’s role:

- **Basic Users:** See their own activity and limited tenant information
- **Admins:** See tenant-wide statistics and user management options
- **Super Admins:** See cross-tenant statistics and tenant management options

User Management (Admin Only)

Administrators can manage users within their tenant:

1. Navigate to the User Management section from the Dashboard.
2. View a list of all users in the tenant.
3. Add new users by clicking “Add User” and filling in the required information.
4. Edit existing users by clicking the edit icon next to a user.
5. Deactivate users by clicking the deactivate icon next to a user.

Tenant Management (Super Admin Only)

Super administrators can manage tenants:

1. Navigate to the Tenant Management section from the Dashboard.
2. View a list of all tenants.
3. Add new tenants by clicking “Add Tenant” and filling in the required information.
4. Edit existing tenants by clicking the edit icon next to a tenant.
5. Change tenant status (active, inactive, suspended, trial) as needed.

Tenant Configuration

Tenant administrators can configure tenant-specific settings:

1. Navigate to the Tenant Configuration section from the Dashboard.
2. View current configuration values.
3. Add or update configuration values by entering a key and value and clicking “Save”.
4. Delete configuration values by clicking the delete icon next to a configuration.

API Usage

The SaaS Platform provides a comprehensive RESTful API that can be used for integration with other systems or for building custom frontends.

API Documentation

The API documentation is available at <http://127.0.0.1:8000/docs> when the backend is running. This interactive documentation allows you to:

- Browse all available endpoints
- See request and response schemas
- Test endpoints directly from the browser
- Authenticate and use protected endpoints

Authentication

To use the API, you need to authenticate and obtain an access token:

1. Make a POST request to `/token` with your credentials:

```
curl -X POST http://127.0.0.1:8000/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=your.email@example.com&password=your-password"
```

2. The response will include an access token and a refresh token:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

3. Use the access token in subsequent requests:

```
curl -X GET http://127.0.0.1:8000/protected-endpoint \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Tenant Identification

All API requests must include tenant identification, either:

- As an `X-Tenant-ID` header:

```
curl -X GET http://127.0.0.1:8000/endpoint \
  -H "Authorization: Bearer your-token" \
  -H "X-Tenant-ID: your-tenant-slug"
```

- Or as a `tenant` query parameter:

```
curl -X GET "http://127.0.0.1:8000/endpoint?tenant=your-tenant-slug" \
  -H "Authorization: Bearer your-token"
```

Key Endpoints

The API provides the following key endpoints:

Authentication Endpoints

- `POST /register`: Register a new user
- `POST /token`: Login and get access token
- `POST /refresh-token`: Refresh an access token
- `POST /logout`: Logout and invalidate token
- `POST /reset-password/request`: Request password reset
- `POST /reset-password/verify`: Verify reset token
- `POST /reset-password/reset`: Reset password with token

Tenant Endpoints

- `POST /tenants`: Create a new tenant (super admin only)
- `GET /tenants`: List all tenants (super admin only)
- `GET /tenants/{tenant_id}`: Get tenant details (super admin only)
- `PUT /tenants/{tenant_id}`: Update tenant (super admin only)

Tenant Configuration Endpoints

- `POST /tenants/{tenant_id}/config`: Create/update tenant config
- `GET /tenants/{tenant_id}/config/{key}`: Get tenant config value
- `GET /tenants/{tenant_id}/config`: List all tenant configs

Statistics Endpoints

- `GET /admin/stats/mau`: Get Monthly Active Users statistics
- `GET /admin/stats/usage`: Get usage statistics
- `GET /admin/stats/user-activity/{user_id}`: Get user activity history
- `GET /super-admin/stats/tenants`: Get tenant statistics (super admin only)

- GET `/super-admin/stats/global-mau`: Get global MAU statistics (super admin only)

Health Check Endpoint

- GET `/health`: Check API health status
- GET `/`: Root endpoint

Multi-tenancy Features

The SaaS Platform implements a comprehensive multi-tenancy model that allows multiple organizations to use the same application instance while keeping their data isolated.

Tenant Model

Each tenant represents a separate organization or customer and has:

- A unique identifier (slug)
- A status (active, inactive, suspended, trial)
- Custom configuration options
- Its own set of users

Tenant Isolation

Data isolation is implemented at the row level:

- All data is associated with a `tenant_id`
- Queries automatically filter by the current tenant
- Users can only access data from their own tenant
- Cross-tenant operations are restricted to super administrators

Tenant Identification

The current tenant is identified in each request through:

- The `X-Tenant-ID` header
- The `tenant` query parameter

The middleware automatically detects and validates the tenant for each request.

Tenant-specific Authentication

Authentication is tenant-specific:

- Users are registered within a specific tenant
- Login credentials are validated against the specified tenant
- JWT tokens include tenant information
- The same email can be used in different tenants (with different accounts)

Tenant Configuration

Each tenant can have custom configuration options stored as key-value pairs:

- Feature flags
- Branding settings
- Integration parameters
- Usage limits

These configurations can be managed through the API or the admin interface.

Authentication and Security

The SaaS Platform implements robust security measures to protect user data and prevent unauthorized access.

Password Security

User passwords are protected with:

- Bcrypt hashing algorithm
- Password complexity requirements:
 - Minimum 8 characters
 - At least 1 uppercase letter
 - At least 1 number
 - At least 1 special character
- Password reset functionality with time-limited tokens

JWT Authentication

The system uses JSON Web Tokens (JWT) for authentication:

- Access tokens with short expiry (30 minutes by default)
- Refresh tokens with longer expiry (7 days by default)
- Token blacklisting to prevent reuse of revoked tokens
- Token refresh mechanism to maintain sessions

Role-Based Access Control

Users have different roles that determine their permissions:

- **Super Admin:** Can manage all tenants and access cross-tenant data
- **Admin:** Can manage users and settings within their tenant
- **Basic User:** Can access standard functionality within their tenant

Request Traceability

All requests are traceable through:

- Unique request IDs assigned to each request

- Comprehensive activity logging
- IP address and user agent tracking
- Structured logging with request correlation

Rate Limiting

The API implements rate limiting to prevent abuse:

- Login attempts are limited to prevent brute force attacks
- Password reset requests are limited to prevent email flooding
- API endpoints have appropriate rate limits based on their sensitivity

Troubleshooting

This section provides solutions to common issues you might encounter when using the SaaS Platform.

Database Connection Issues

Symptom: The application fails to start with database connection errors.

Solutions: 1. Verify that PostgreSQL is running: “bash # Ubuntu/Debian
sudo service postgresql status

macOS brew services list | grep postgresql

Windows sc query postgresql “

2. Check that the database exists:

```
sudo -u postgres psql -c "\l" | grep saas_platform
```

3. Verify the database credentials in the .env file.

4. Ensure the database user has the necessary permissions:

```
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE saas_platform TO saas_user;"
```

Redis Connection Issues

Symptom: The application starts but shows Redis connection warnings.

Solutions: 1. Verify that Redis is running: “bash # Ubuntu/Debian sudo
service redis-server status

macOS brew services list | grep redis

Windows sc query redis “

2. Check the Redis connection string in the .env file.

3. Test the Redis connection:

```
redis-cli ping
```

API Startup Issues

Symptom: The API fails to start.

Solutions: 1. Check for error messages in the terminal output.

2. Verify that all dependencies are installed:

```
pip install -r requirements.txt
```

3. Ensure the required ports are available:

```
# Check if port 8000 is in use
netstat -tuln | grep 8000
```

4. Verify the Python version:

```
python --version
```

Streamlit UI Issues

Symptom: The Streamlit UI fails to start or connect to the API.

Solutions: 1. Ensure the API is running before starting Streamlit.

2. Check for error messages in the Streamlit terminal.
3. Verify that the API URL in the Streamlit app is correct (default is `http://127.0.0.1:8000`).
4. Check if port 8501 is available:

```
netstat -tuln | grep 8501
```

Authentication Issues

Symptom: Unable to log in or access protected endpoints.

Solutions: 1. Verify that the `SECRET_KEY` in the `.env` file is set correctly.

2. Check that the tenant is active and exists.
3. Verify user credentials.
4. Ensure that the token is being sent correctly in the Authorization header.
5. Check if the token has expired and needs to be refreshed.

Email Sending Issues

Symptom: Password reset emails are not being sent.

Solutions: 1. Verify the SMTP settings in the `.env` file.

2. Check if the SMTP server requires authentication.
3. Test the SMTP connection:

```
telnet smtp.example.com 587
```

4. Check for firewall restrictions that might block outgoing SMTP connections.

Frequently Asked Questions

General Questions

Q: Can I run the API and Streamlit UI on different servers?

A: Yes, you can run them on different servers. Just make sure to update the API URL in the Streamlit app to point to the correct server.

Q: How do I create the first super admin user?

A: The first super admin user needs to be created directly in the database. You can use the following SQL:

```
INSERT INTO tenants (name, slug, status) VALUES ('System', 'system', 'active');
INSERT INTO users (email, password_hash, role, tenant_id)
VALUES ('admin@example.com', 'hashed_password', 'super_admin', 1);
```

Replace 'hashed_password' with a properly hashed password using the bcrypt algorithm.

Q: Can I customize the branding of the Streamlit UI?

A: Yes, you can customize the Streamlit UI by modifying the `streamlit_app.py` file. Streamlit provides various options for theming and layout customization.

Multi-tenancy Questions

Q: How is data isolated between tenants?

A: Data is isolated at the row level with `tenant_id` foreign keys. All queries include tenant filtering to ensure users can only access data from their own tenant.

Q: Can users belong to multiple tenants?

A: In the current implementation, a user can only belong to one tenant. However, the same email address can be used to create separate accounts in different tenants.

Q: How do I create a new tenant?

A: New tenants can be created by super admins through the API endpoint `POST /tenants` or through the admin interface in the Streamlit UI.

Security Questions

Q: How are passwords stored?

A: Passwords are hashed using the bcrypt algorithm, which is a secure one-way hashing function designed specifically for password storage.

Q: How long are access tokens valid?

A: Access tokens are valid for 30 minutes by default. This can be configured using the `ACCESS_TOKEN_EXPIRE_MINUTES` environment variable.

Q: How can I implement additional security measures?

A: You can enhance security by: - Enabling HTTPS for API and UI - Implementing IP-based access restrictions - Adding two-factor authentication - Configuring more restrictive CORS settings - Implementing audit logging

Performance Questions

Q: How many concurrent users can the system handle?

A: The system's capacity depends on your hardware configuration. With the default settings, it should handle hundreds of concurrent users on modest hardware.

Q: How can I improve performance for a large number of users?

A: To improve performance: - Increase database connection pool size - Set up database read replicas - Implement caching with Redis - Deploy the API behind a load balancer - Scale horizontally with multiple API instances

Q: Is the system suitable for production use?

A: Yes, the system is designed with production use in mind. However, before deploying to production, you should: - Enable HTTPS - Configure proper logging - Set up monitoring - Implement regular backups - Review and harden security settings